

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

Annexure A

INDUSTRY PROBLEM SOLVING TASK

Code of Conduct:

I, **Ashira Bridget Susan S (192524316)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Ashira Bridget Susan S', is written below the signature line.

S.No.	Industry Problem-Solving Task	Marks
1	A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.	10
2	A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm (FCFS, SSTF, SCAN) to reduce seek time and explain how it improves performance in this scenario.	10
3	An operating system uses a bitmap for free-space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.	10
4	A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.	10
5	A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.	10

Intelligent File Allocation and Storage Optimization System

Industry Context:

Cloud Storage / Data Centers

Design a file-storage simulator for a data center that supports **Contiguous, Sequential/Linked, and Indexed allocation**.

Problem Requirements:

- | | | |
|---|---|----|
| 6 | <ul style="list-style-type: none"> • Accept file creation requests with file name and required number of blocks. • Implement all three allocation strategies. • Maintain a dynamic disk-block map. • Support file creation, deletion, extension, and block release. • Detect insufficient storage. • Calculate internal and external fragmentation. • Compare storage utilization and access efficiency. | 10 |
|---|---|----|

S.No.	Industry Problem-Solving Task	Marks
-------	-------------------------------	-------

- Recommend the most suitable allocation strategy based on file size and access pattern.

Advanced Challenge:

Develop an algorithm that automatically selects the allocation strategy for each file based on its size, growth rate, and sequential/random access requirements.

Expected Outcome:

A storage-management dashboard showing block allocation, fragmentation, utilization, and recommended allocation method.

Unix File System Inode and Buffer Cache Simulator

Industry Context:

Operating-System Kernel / Storage Systems

Develop a simulator that models the internal operation of a Unix-like file system using inodes and buffer cache.

Problem Requirements:

- Maintain an inode table containing file metadata.
- Implement simulated versions of:
 - o ialloc()
 - o ifree()
 - o namei()
 - o alloc()
 - o free()
- Implement pathname resolution using namei().
- Maintain a buffer cache for disk blocks.
- Simulate cache hit and cache miss situations.
- Track dirty and clean buffers.
- Implement buffer replacement when the cache is full.
- Calculate cache hit ratio and disk-access reduction.

Advanced Challenge:

S.No.	Industry Problem-Solving Task	Marks
-------	-------------------------------	-------

Compare different buffer-cache replacement strategies and determine which provides the best performance for different workloads.

Expected Outcome:

A simulation showing the complete path:

Application → System Call → Inode → Buffer Cache → Disk

Distributed Network File System with Mount/Unmount Management

Industry Context:

Enterprise Servers / Distributed Storage

Design a Network File System (NFS)-like simulator where multiple client machines access files stored on a remote server.

Problem Requirements:

- Create a centralized file server.
- Simulate multiple clients.
- Implement file creation, opening, reading, writing, and deletion.
- Implement mounting and unmounting of remote file systems.
- Maintain client-side mount tables.
- Handle simultaneous file access.
- Simulate server failure and recovery.
- Detect invalid mount/unmount operations.
- Maintain file consistency between clients and server.

8

10

Advanced Challenge:

Design a mechanism to handle concurrent writes to the same file while minimizing conflicts and maintaining consistency.

Expected Outcome:

A distributed file-system simulator showing:

Client → Network → File Server → File System → Disk

Intelligent File-System Recovery and Crash Consistency System

9

10

Industry Context:

S.No.	Industry Problem-Solving Task	Marks
-------	-------------------------------	-------

Database Servers / Banking / Mission-Critical Systems Develop a simulator that investigates what happens when a system crashes during file-system operations.

Problem Requirements:

- Simulate file creation, deletion, allocation, and modification.
- Maintain metadata such as inodes and free-block information.
- Generate random system crashes during operations.
- Identify inconsistent states after a crash.
- Detect:
 - o Lost blocks
 - o Duplicate block allocation
 - o Invalid inode references
 - o Orphan files
 - o Corrupted free-space information
- Implement a recovery mechanism.
- Restore the file system to a consistent state.

Advanced Challenge:

Compare different recovery approaches and calculate data loss, recovery time, and consistency level.

Expected Outcome:

A File System Health and Recovery Tool that reports:

Normal State → Crash → Corruption Detection → Recovery → Verified File System

High-Performance I/O and File-System Optimization Simulator

Industry Context:

10	Cloud Platforms / Video Streaming / High-Performance Computing	10
-----------	--	-----------

Develop a simulator that optimizes file-system performance for a workload involving thousands of file operations.

Problem Requirements:

S.No.	Industry Problem-Solving Task	Marks
-------	-------------------------------	-------

- Generate read/write requests from multiple applications.
- Simulate different file sizes and access patterns.
- Implement file allocation and disk-block management.
- Simulate the I/O subsystem and request queues.
- Include buffer caching.
- Measure:
 - o File access time
 - o Disk I/O operations
 - o Cache hit ratio
 - o Throughput
 - o Storage utilization
 - o Fragmentation
- Compare sequential and random file access.
- Identify system bottlenecks.

Advanced Challenge:

Build an adaptive optimization algorithm that dynamically changes caching, allocation, and I/O scheduling decisions according to the workload.

Expected Outcome:

A performance analyzer that recommends the optimal configuration for workloads such as:

- Video streaming
- Database transactions
- Web-server logs
- Large scientific datasets
- Cloud file storage

OPERATING SYSTEMS – CASE STUDY SOLUTIONS

File Allocation, Disk Scheduling and Free-Space Management

1. Selecting the Most Suitable File Allocation Method

Problem Statement

A cloud storage company is facing performance issues due to frequent file access delays. The company needs to determine whether **contiguous, linked, or indexed allocation** would be most suitable for improving file access speed.

Analysis

File allocation determines how the blocks of a file are stored on a disk. The three common methods are:

1. Contiguous Allocation

In contiguous allocation, all blocks of a file are stored in consecutive disk locations.

Advantages:

- Provides very fast sequential access.
- Provides excellent direct/random access because the address of any block can be calculated easily.
- Requires fewer disk head movements.
- Improves cache and read performance because consecutive blocks are physically close.

Disadvantages:

- External fragmentation may occur.
- File growth can be difficult because adjacent blocks may not be available.
- May require moving the entire file when sufficient contiguous space is unavailable.

2. Linked Allocation

In linked allocation, file blocks can be stored anywhere on the disk. Each block contains a pointer to the next block.

Advantages:

- Avoids external fragmentation.
- Files can grow easily.
- Efficient for sequential access.

Disadvantages:

- Direct/random access is slow because blocks must be followed sequentially.
- Pointer storage introduces additional overhead.
- Blocks may be physically scattered, increasing disk access time.

3. Indexed Allocation

In indexed allocation, an index block stores the addresses of all blocks belonging to a file.

Advantages:

- Supports direct/random access.
- Files do not need to occupy contiguous disk locations.
- Avoids external fragmentation.

Disadvantages:

- Requires additional space for index blocks.
- Accessing a file may require an additional index-block access.
- Very large files may require multiple levels of indexing.

Recommended Solution: Contiguous Allocation

For a cloud storage system where **frequent file access speed** is the primary concern, **contiguous allocation** is the most suitable choice, provided that sufficient storage management techniques are used to handle fragmentation.

Justification

Contiguous allocation stores file blocks next to one another. Therefore, when a file is accessed, the disk can read consecutive blocks with minimal head movement. It also provides excellent random-access performance because the location of a particular block can be calculated directly.

For example, if a file begins at block 100, its fifth block can be accessed directly using the starting address and block offset. This significantly reduces access delays.

Conclusion

Contiguous allocation is the preferred choice when the primary objective is maximum file access speed. Although it has limitations such as external fragmentation and difficulty in file expansion, its excellent sequential and random-access performance makes it appropriate for a performance-oriented cloud storage environment.

Final Recommendation: CONTIGUOUS ALLOCATION

2. Selecting the Appropriate Disk Scheduling Algorithm

Problem Statement

A data center experiences high disk head movement while processing user requests. The company needs to select the most appropriate algorithm among **FCFS**, **SSTF**, and **SCAN** to reduce seek time and improve performance.

Analysis of Algorithms

1. FCFS – First Come First Serve

FCFS processes disk requests in the exact order in which they arrive.

Advantages:

- Simple to implement.
- Fair because requests are handled in arrival order.
- No starvation.

Disadvantages:

- Can cause significant disk head movement.
- Does not consider the current position of the disk head.
- Can result in high average seek time.

2. SSTF – Shortest Seek Time First

SSTF selects the pending request that is closest to the current disk-head position.

Advantages:

- Generally reduces immediate seek distance.
- Provides better average seek time than FCFS.
- Improves disk utilization.

Disadvantages:

- Requests located far away may experience starvation.
- The disk head may repeatedly serve requests in one region.

3. SCAN – Elevator Algorithm

SCAN moves the disk head in one direction, servicing requests along the way. When it reaches the end of the disk, it reverses direction and services requests in the opposite direction.

Advantages:

- Reduces unnecessary disk-head movement.

- Provides more predictable service than SSTF.
- Handles a large number of requests efficiently.
- Reduces the possibility of starvation.
- Provides a good balance between performance and fairness.

Recommended Solution: SCAN

For a **data center with high disk head movement and a large number of requests**, **SCAN is the most appropriate choice.**

Justification

SCAN works similarly to an elevator. Instead of moving randomly between requests, the disk head travels systematically in one direction and services all pending requests encountered along its path.

For example, if the disk head is moving from track 50 toward track 100, it can service requests at tracks 55, 65, 80, and 95 during the same movement. After reaching the end, it reverses direction and services requests on the other side.

This reduces unnecessary back-and-forth movement and provides a lower and more consistent average seek time.

Why Not FCFS?

FCFS can cause the disk head to move repeatedly between distant tracks. This results in excessive seek time and poor performance when many requests are present.

Why Not SSTF?

SSTF can provide low seek time for individual requests, but it may repeatedly select nearby requests and delay requests located farther away. In a busy data center, this can result in starvation.

Conclusion

SCAN is the most suitable algorithm because it efficiently handles heavy disk workloads while reducing disk-head movement and maintaining reasonable fairness among requests.

Final Recommendation: SCAN

3. Optimizing Bitmap-Based Free-Space Management

Problem Statement

An operating system uses a bitmap for free-space management but experiences slow allocation when the disk size increases. An optimization technique is required to improve bitmap-based allocation.

Background

A bitmap uses one bit to represent each disk block.

For example:

- 0 → Block is free
- 1 → Block is allocated

If a disk contains 1,000,000 blocks, the bitmap must contain 1,000,000 bits. As the disk becomes larger, searching the entire bitmap for free blocks becomes increasingly time-consuming.

Problem with Basic Bitmap Allocation

A simple bitmap allocation algorithm may scan the bitmap from the beginning until it finds a free block or a group of consecutive free blocks.

For a large disk, this can result in:

- Long bitmap searches.
- Increased CPU overhead.
- Slower file allocation.
- Poor performance when free blocks are scattered.

Recommended Optimization: Hierarchical Bitmap

A **hierarchical or multi-level bitmap** can be used to improve allocation efficiency.

Instead of searching the entire bitmap, the bitmap can be divided into smaller groups of bits. A higher-level bitmap records whether each group contains free blocks.

For example:

Level 1: Indicates whether a large group contains any free blocks.

Level 2: Identifies the smaller subgroup containing free blocks.

Level 3: Identifies the exact free block.

How It Works

Suppose the disk contains thousands of blocks.

Instead of scanning every bit:

1. The operating system first checks the higher-level bitmap.
2. It identifies a group containing available space.
3. It moves to the corresponding lower-level bitmap.
4. It searches only that smaller section.
5. The required free block is located much faster.

Advantages

- Significantly reduces bitmap search time.
- Scales better for large disks.
- Reduces CPU overhead.
- Makes free-block allocation faster.
- Allows efficient identification of contiguous free blocks.
- Maintains the simplicity of bitmap-based management.

Example

Consider a disk containing **1,000,000 blocks**.

A traditional bitmap may require a search through a large portion of the bitmap to locate free space.

With hierarchical indexing, the system can first identify a group containing free blocks and then search only that group.

Therefore:

Traditional Bitmap:

Entire Bitmap → Search → Find Free Block

Hierarchical Bitmap:

Higher-Level Bitmap → Free Group → Lower-Level Bitmap → Free Block

This reduces the amount of unnecessary searching.

Conclusion

A **hierarchical bitmap** is an effective optimization for large disks. It preserves the advantages of bitmap-based free-space management while reducing the time required to locate available blocks.

Final Recommendation: USE A HIERARCHICAL / MULTI-LEVEL BITMAP

4. Choosing Between SCAN and C-SCAN for a Real-Time Application

Problem Statement

A real-time application requires consistent and predictable disk access times. The system must choose between **SCAN** and **C-SCAN** based on its performance requirements.

SCAN

SCAN moves the disk head in one direction, servicing requests along the way. When it reaches the end of the disk, it reverses direction and services requests while moving back.

Advantages:

- Reduces unnecessary disk-head movement.
- Provides good average seek time.
- Provides better fairness than SSTF.
- Works efficiently under heavy workloads.

Limitation:

Requests near the middle of the disk may receive service more frequently than requests near the edges, depending on the workload and head position.

C-SCAN – Circular SCAN

C-SCAN services requests while moving in one direction only. After reaching the end of the disk, the head returns to the beginning without servicing requests during the return movement. It then starts servicing requests again in the same direction.

Advantages of C-SCAN

- Provides more uniform waiting times.
- Offers predictable request servicing.
- Treats disk requests more uniformly.
- Is particularly useful when consistent response time is more important than maximizing individual request efficiency.

Comparison

Feature	SCAN	C-SCAN
Head movement	Both directions	One servicing direction
Fairness	Good	Very good
Waiting-time uniformity	Moderate	High

Feature	SCAN	C-SCAN
Predictability	Good	Better
Suitable for real-time workloads	Good	Excellent
Request servicing	During both directions	During one direction

Recommended Solution: C-SCAN

For a **real-time application requiring consistent and predictable disk access times**, **C-SCAN is more suitable than SCAN**.

Justification

In SCAN, the disk head services requests while moving in both directions. Therefore, the waiting time for a request can vary depending on where it is located and the current direction of the head.

C-SCAN, on the other hand, services requests in a single direction. Once the head reaches the end, it returns to the beginning and starts another servicing cycle.

This creates a more uniform waiting-time distribution and makes response times easier to predict.

Example

Suppose the disk head services requests from lower-numbered tracks toward higher-numbered tracks.

With **SCAN**:

Forward → Service Requests → Reverse → Service Requests

With **C-SCAN**:

Forward → Service Requests → Return to Beginning → Forward → Service Requests

C-SCAN therefore provides a more consistent service pattern.

Conclusion

Since the application is **real-time** and requires **predictable and consistent access times**, C-SCAN is the better choice. Its circular servicing pattern provides more uniform waiting times and reduces variation in request response.

Final Recommendation: C-SCAN

5. Designing a Hybrid File Allocation and Disk Scheduling System

Problem Statement

A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. The system must efficiently handle large-scale data access.

The company needs to select a suitable combination of:

File Allocation + Disk Scheduling

Recommended Combination

Indexed Allocation + SCAN Scheduling

Why Indexed Allocation?

Indexed allocation stores the addresses of a file's data blocks in an index block.

This is particularly useful for large-scale storage because files do not need to occupy consecutive disk blocks.

Advantages:

- Supports direct/random access.
- Avoids external fragmentation.
- Allows files to grow dynamically.
- Suitable for large files.
- Data blocks can be distributed across available disk space.

For a large-scale storage system, these characteristics are valuable because storage requirements can change frequently.

Why SCAN Scheduling?

SCAN moves the disk head systematically across the disk while servicing requests.

Advantages:

- Reduces unnecessary disk-head movement.
- Provides good average seek time.
- Handles heavy workloads efficiently.
- Reduces starvation compared with SSTF.
- Provides a good balance between performance and fairness.

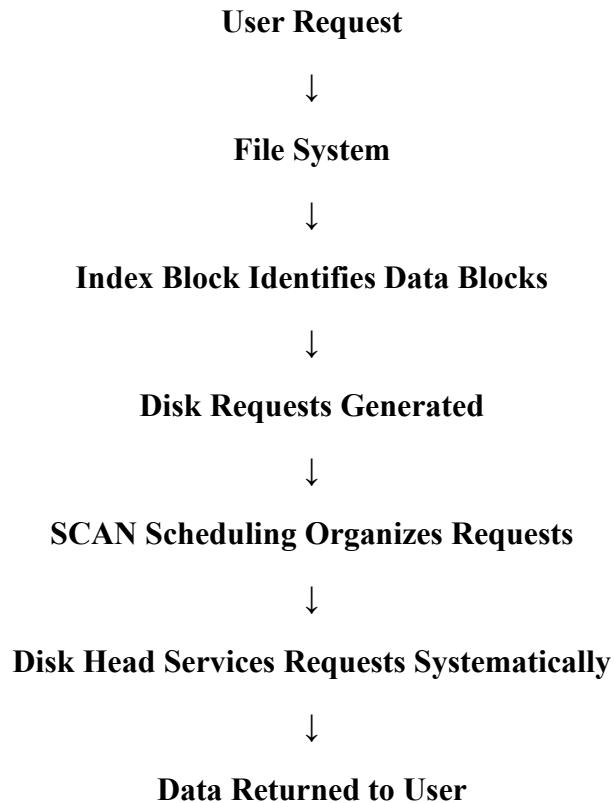
How the Hybrid System Works

The two techniques solve different problems.

Indexed Allocation determines **where the blocks of each file are stored**.

SCAN Scheduling determines **the order in which disk requests are serviced**.

The overall process can be represented as:



Example Scenario

Suppose a company stores large datasets consisting of thousands of files.

A particular file may have its blocks stored at:

120 → 450 → 452 → 700 → 905

Because indexed allocation stores these block addresses in an index structure, the operating system can locate the required blocks without requiring them to be contiguous.

When multiple disk requests are generated, SCAN organizes them according to the current movement of the disk head.

For example, if the head is moving toward higher-numbered tracks, requests encountered along that direction can be serviced first instead of repeatedly moving back and forth across the disk.

Performance Benefits

The combination provides several benefits:

1. Efficient Large-File Storage

Indexed allocation supports large files without requiring continuous free space.

2. Reduced Fragmentation Problems

Files can use available blocks distributed across the disk.

3. Better Disk Utilization

SCAN minimizes unnecessary movement while servicing multiple requests.

4. Improved Scalability

The system can accommodate growing data volumes without depending on large contiguous regions.

5. Balanced Performance

Indexed allocation provides flexible data placement, while SCAN improves the efficiency of physical disk access.

Why Not Contiguous + SCAN?

Contiguous allocation provides excellent access speed, but large-scale systems may suffer from external fragmentation and difficulty expanding files.

Why Not Linked + SSTF?

Linked allocation is good for sequential access but is inefficient for random access. SSTF can also cause starvation when some requests remain far from the current disk-head position.

Why Indexed + SCAN Is Suitable

For a large-scale enterprise storage environment, **flexibility and predictable disk servicing** are important. Indexed allocation provides flexible file placement and direct access, while SCAN provides systematic disk-head movement.

Therefore, the combination offers a practical balance between **storage flexibility, access efficiency, scalability, fairness, and disk performance**.

Conclusion

The most suitable hybrid approach is:

INDEXED ALLOCATION + SCAN SCHEDULING

Indexed allocation efficiently manages large and dynamically growing files, while SCAN reduces unnecessary disk-head movement and handles a high volume of requests efficiently.

Final Recommendation: INDEXED ALLOCATION + SCAN

6. INTELLIGENT FILE ALLOCATION AND STORAGE OPTIMIZATION SYSTEM

Industry Context

Cloud Storage / Data Centers

6.1 Problem Statement

Modern cloud storage systems handle a large number of files with different sizes, growth patterns, and access requirements. Selecting an inappropriate file-allocation method can result in fragmentation, inefficient storage utilization, and increased access time.

The proposed **Intelligent File Allocation and Storage Optimization System** is a file-storage simulator that implements and compares **Contiguous, Linked, and Indexed Allocation** strategies.

The system dynamically manages disk blocks and analyzes the performance of each allocation strategy. Based on file size, expected growth, and access pattern, it can automatically recommend the most suitable allocation method.

6.2 Objectives

The main objectives of the system are:

1. To simulate different file-allocation techniques.
 2. To maintain a dynamic representation of disk blocks.
 3. To support file creation, deletion, extension, and block release.
 4. To detect insufficient storage conditions.
 5. To calculate internal and external fragmentation.
 6. To calculate storage utilization.
 7. To compare access efficiency among allocation methods.
 8. To recommend a suitable allocation strategy for different files.
 9. To provide an interactive storage-management dashboard.
 10. To automatically select an allocation strategy based on file characteristics.
-

6.3 Allocation Strategies

The simulator implements three major allocation strategies.

A. Contiguous Allocation

In contiguous allocation, all blocks belonging to a file are stored consecutively.

For example:

File A → [10] [11] [12] [13] [14]

Advantages

- Very fast sequential access.
- Very fast direct/random access.
- Simple block-address calculation.
- Low access overhead.

Disadvantages

- External fragmentation can occur.
 - File expansion may be difficult.
 - A sufficiently large continuous free region is required.
-

B. Linked Allocation

In linked allocation, file blocks can be located anywhere on the disk. Each block contains a pointer to the next block.

Example:

File B → [4] → [17] → [8] → [25] → NULL

Advantages

- No requirement for contiguous free blocks.
- Files can easily grow.
- External fragmentation is avoided.
- Suitable for sequential access.

Disadvantages

- Random access is slow.
 - Pointer storage creates overhead.
 - Blocks may be physically scattered.
 - More disk accesses may be required.
-

C. Indexed Allocation

In indexed allocation, a separate index block stores the addresses of the file's data blocks.

Example:

Index Block → [6, 14, 22, 31]

Therefore:

File C → **Index Block** → **Data Blocks**

Advantages

- Supports direct/random access.
- Files do not need contiguous storage.
- External fragmentation is avoided.
- Files can grow dynamically.

Disadvantages

- Requires additional index-block space.
 - Large files may require multiple index blocks.
 - Index access introduces additional overhead.
-

6.4 System Modules

The proposed simulator can be divided into the following modules:

Module 1 – File Request Manager

Accepts:

- File name
- Required number of blocks
- Access pattern
- Expected growth rate
- Allocation strategy

Example:

File Name: CustomerData

Blocks Required: 20

Access Pattern: Sequential

Growth Rate: High

Module 2 – Allocation Engine

The allocation engine implements:

- Contiguous allocation
- Linked allocation
- Indexed allocation

It checks the disk map and assigns available blocks according to the selected strategy.

Module 3 – Dynamic Disk-Block Manager

The disk is represented using a dynamic block map.

Example:

Block	Status	File
0	Free	—
1	Used	File A
2	Used	File A
3	Free	—
4	Used	File B
5	Free	—

The map is updated whenever files are created, extended, or deleted.

Module 4 – File Operations Manager

The simulator supports:

Create → Allocate Blocks

Extend → Allocate Additional Blocks

Delete → Release Blocks

Free → Return Blocks to Disk

Module 5 – Fragmentation Analyzer

The system calculates:

- Internal fragmentation
 - External fragmentation
 - Free-space percentage
 - Number of free blocks
 - Number of fragmented regions
-

Module 6 – Performance Analyzer

The analyzer compares:

- Storage utilization
 - Access time
 - Number of disk accesses
 - Allocation overhead
 - Fragmentation
 - File-growth efficiency
-

Module 7 – Intelligent Recommendation Engine

The recommendation engine determines the most appropriate allocation method based on:

File Size + Growth Rate + Access Pattern

Example:

File Characteristics	Recommended Method
Small + Random Access	Indexed
Large + Sequential Access	Contiguous
Frequently Growing File	Linked
Large + Random Access	Indexed

6.5 Advanced Automatic Selection Algorithm

The system can assign a score to each allocation method.

Example:

Contiguous Score

- High sequential access $\rightarrow +3$
- High random access $\rightarrow +3$
- Large file $\rightarrow +2$
- Low growth rate $\rightarrow +2$

Linked Score

- Sequential access $\rightarrow +3$
- High growth rate $\rightarrow +3$
- Large file $\rightarrow +2$
- Random access $\rightarrow -2$

Indexed Score

- Random access $\rightarrow +3$
- Large file $\rightarrow +3$
- High growth rate $\rightarrow +2$
- Sequential-only access $\rightarrow +1$

The method with the highest score is recommended.

Example

Suppose:

File Size: Large

Growth Rate: Low

Access Pattern: Sequential

The system may calculate:

Contiguous = 10

Linked = 6

Indexed = 7

Therefore:

Recommended Method $\rightarrow$ Contiguous Allocation

6.6 Fragmentation Calculation

External Fragmentation

External fragmentation can be calculated as:

$$\text{External Fragmentation} = \text{Total Free Space} - \text{Largest Contiguous Free Space}$$

Example:

Total free space = 30 blocks

Largest continuous free region = 18 blocks

Therefore:

$$\text{External Fragmentation} = 30 - 18 = 12 \text{ blocks}$$

Storage Utilization

Storage utilization can be calculated as:

$$\text{Storage Utilization} = (\text{Used Blocks} / \text{Total Blocks}) \times 100$$

Example:

Used blocks = 700

Total blocks = 1000

$$\text{Storage Utilization} = 70\%$$

6.7 Expected Dashboard

The simulator dashboard can display:

Total Blocks: 1000

Used Blocks: 720

Free Blocks: 280

Storage Utilization: 72%

External Fragmentation: 35 blocks

Number of Files: 48

It can also display a visual block map:

[A][A][A][][][B][B][][C][C][C][][D][][E]

where different symbols represent different files and free blocks.

6.8 Expected Outcome

The final system should provide a storage-management dashboard showing:

- Current disk-block allocation.
 - File details.
 - Free and occupied blocks.
 - Fragmentation statistics.
 - Storage utilization.
 - Access-efficiency comparison.
 - Recommended allocation strategy.
 - Allocation/deallocation history.
-

6.9 Conclusion

The **Intelligent File Allocation and Storage Optimization System** provides a practical simulation of how operating systems manage files on storage devices.

By combining allocation simulation, fragmentation analysis, performance measurement, and intelligent recommendation, the system demonstrates how different allocation strategies affect storage performance.

Final Outcome:

File Request → Allocation Decision → Block Allocation → Performance Analysis → Intelligent Recommendation

7. UNIX FILE SYSTEM INODE AND BUFFER CACHE SIMULATOR

Industry Context

Operating-System Kernel / Storage Systems

7.1 Problem Statement

Unix-like operating systems use **inodes** to store file metadata and **buffer caches** to reduce repeated disk accesses.

Understanding the interaction between these components is important for analyzing operating-system performance.

The proposed **Unix File System Inode and Buffer Cache Simulator** models the internal operation of a Unix-like file system and demonstrates how application requests travel through system calls, inodes, buffer cache, and disk storage.

7.2 Objectives

The main objectives are:

1. To simulate an inode table.
2. To store file metadata using inodes.
3. To implement simulated versions of `ialloc()`, `ifree()`, `namei()`, `alloc()`, and `free()`.
4. To simulate pathname resolution.
5. To implement a buffer cache.
6. To simulate cache hit and cache miss situations.
7. To maintain dirty and clean buffers.
8. To implement buffer replacement.
9. To calculate cache hit ratio.
10. To measure reduction in disk accesses.
11. To compare different cache replacement strategies.

7.3 Inode Structure

An inode stores metadata associated with a file.

A simulated inode may contain:

Field	Description
Inode Number	Unique identifier
File Type	File or directory
File Size	Size of the file
Permissions	Access permissions
Owner	File owner
Block Addresses	Data-block locations
Link Count	Number of references
Status	Free or allocated

Example:

Inode 12

File Name: report.txt

File Size: 4096 bytes

Permissions: rw-r--r--

Blocks: 20, 21, 22, 23

Status: Allocated

7.4 Simulated File-System Functions

A. ialloc()

ialloc() allocates a free inode for a newly created file.

Process:

Request → Search Free Inode → Allocate Inode → Initialize Metadata

B. ifree()

ifree() releases an inode when a file is deleted.

Process:

Delete File → Release Data Blocks → Release Inode

C. namei()

namei() converts a pathname into the corresponding inode.

Example:

/home/user/report.txt



Directory Search



Inode Number 15



Inode 15

This allows the operating system to identify the file associated with a pathname.

D. alloc()

alloc() allocates a free disk block.

Process:

Search Free Block → Mark Block Used → Return Block Number

E. free()

free() releases a previously allocated disk block.

Process:

File Deleted → Block Released → Mark Block Free

7.5 Buffer Cache

The buffer cache stores recently accessed disk blocks in main memory.

Example:

Buffer	Block	Status
B1	10	Clean
B2	25	Dirty

Buffer	Block	Status
B3	40	Clean
B4	52	Dirty

Clean Buffer

A clean buffer contains data that matches the corresponding disk block.

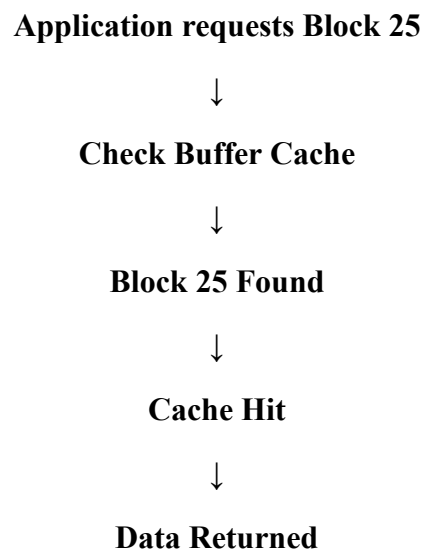
Dirty Buffer

A dirty buffer contains modified data that has not yet been written back to disk.

7.6 Cache Hit

A cache hit occurs when the requested block is already present in the buffer cache.

Example:

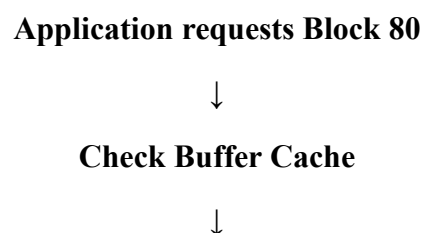


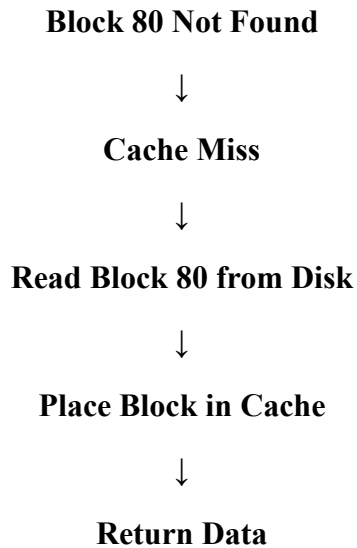
No additional disk read is required.

7.7 Cache Miss

A cache miss occurs when the requested block is not present in the buffer cache.

Process:





7.8 Cache Hit Ratio

The cache hit ratio measures how frequently requested blocks are found in the cache.

Formula:

$$\text{Cache Hit Ratio} = \text{Cache Hits} / \text{Total Requests} \times 100$$

Example:

Cache hits = 80

Total requests = 100

$$\text{Cache Hit Ratio} = 80\%$$

A higher cache hit ratio generally indicates better cache performance.

7.9 Buffer Replacement

When the buffer cache becomes full, one existing buffer must be replaced.

The simulator can implement different replacement strategies.

FIFO

The oldest buffer is removed first.

LRU

The least recently used buffer is removed first.

CLOCK

A circular pointer is used to identify a suitable buffer for replacement.

7.10 Advanced Challenge – Replacement Strategy Comparison

The simulator can run the same workload using:

FIFO → Measure Hit Ratio

LRU → Measure Hit Ratio

CLOCK → Measure Hit Ratio

Example:

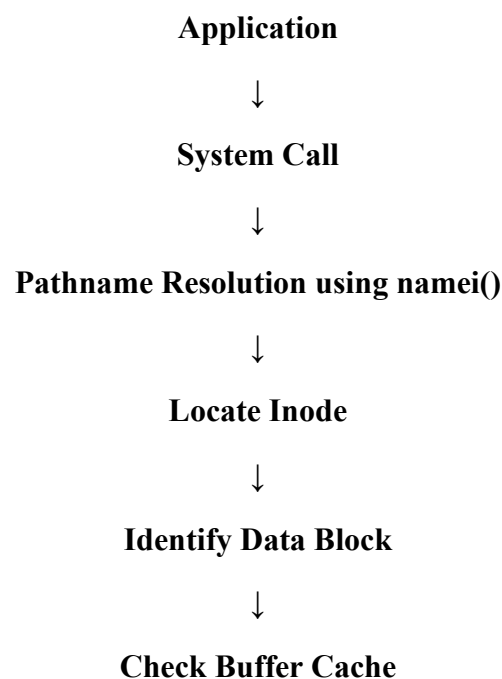
Strategy	Cache Hits	Cache Misses	Hit Ratio
FIFO	68	32	68%
LRU	82	18	82%
CLOCK	78	22	78%

Based on the results, LRU would provide the best performance for this particular workload.

However, the simulator should test multiple workloads because the best strategy can vary according to access patterns.

7.11 Complete System Flow

The complete simulated path is:





Cache Hit?

YES → Return Data

NO → Read from Disk → Update Cache → Return Data

This demonstrates the relationship between the application, file system, memory cache, and physical storage.

7.12 Disk-Access Reduction

The system can calculate disk-access reduction using:

Disk Access Reduction = (Total Requests – Disk Accesses) / Total Requests × 100

Example:

Total requests = 1000

Disk accesses = 250

Therefore:

Disk Access Reduction = 75%

This indicates that the buffer cache eliminated 75% of potential disk accesses.

7.13 Expected Output

The simulator can display:

INODE TABLE

Inode 1 → /home

Inode 2 → /home/user

Inode 3 → /home/user/data.txt

BUFFER CACHE

Block 10 → Clean

Block 20 → Dirty

Block 35 → Clean

CACHE STATISTICS

Total Requests: 1000

Cache Hits: 820

Cache Misses: 180

Cache Hit Ratio: 82%

Disk Accesses: 180
Disk Access Reduction: 82%

7.14 Expected Outcome

The simulator demonstrates the complete interaction:

Application → System Call → Inode → Buffer Cache → Disk

It allows users to observe:

- Inode allocation.
 - Inode release.
 - Pathname resolution.
 - Block allocation.
 - Block release.
 - Cache hits.
 - Cache misses.
 - Dirty buffers.
 - Clean buffers.
 - Buffer replacement.
 - Cache performance.
-

7.15 Conclusion

The **Unix File System Inode and Buffer Cache Simulator** provides a simplified but practical representation of internal Unix-like file-system operations.

It demonstrates how metadata is managed through inodes and how buffer caching reduces expensive disk operations.

The simulator also allows different replacement strategies to be compared and provides quantitative performance measurements such as **cache hit ratio and disk-access reduction**.

Final System Flow:

Application → System Call → Inode → Buffer Cache → Disk

8. DISTRIBUTED NETWORK FILE SYSTEM WITH MOUNT/UNMOUNT MANAGEMENT

Industry Context

Enterprise Servers / Distributed Storage

8.1 Problem Statement

Modern enterprises often allow multiple client machines to access files stored on centralized remote servers. A Network File System (NFS)-like architecture allows users to access remote files as though they were part of the local file system.

The proposed **Distributed Network File System with Mount/Unmount Management** simulates communication between multiple clients, a centralized file server, the server-side file system, and disk storage.

The system also models mounting, unmounting, concurrent access, server failures, recovery, and consistency management.

8.2 Objectives

The main objectives are:

1. To create a centralized file server.
 2. To simulate multiple client machines.
 3. To implement remote file creation.
 4. To implement file opening.
 5. To implement file reading.
 6. To implement file writing.
 7. To implement file deletion.
 8. To simulate mount and unmount operations.
 9. To maintain client-side mount tables.
 10. To handle simultaneous file access.
 11. To simulate server failure and recovery.
 12. To detect invalid mount and unmount operations.
 13. To maintain file consistency between clients and server.
 14. To implement a mechanism for concurrent writes.
-

8.3 System Architecture

The proposed system follows the architecture:

Client 1

↘

Client 2 → Network → File Server → File System → Disk

↗

Client 3

The server acts as the centralized storage provider, while clients send file-system requests through the simulated network.

8.4 Main System Components

Module 1 – File Server

The file server manages:

- Files
- Directories
- File metadata
- Client requests
- File locks
- Storage blocks
- Server status

Example:

Server Status: ONLINE

Files:

- /shared/report.txt
- /shared/data.csv
- /shared/project.doc

Module 2 – Client Simulator

Multiple clients can be created.

Example:

Client 1 – Alice

Client 2 – Bob

Client 3 – Admin

Each client maintains its own mount table and sends requests to the central server.

8.5 Mount Operation

Mounting connects a remote file system to a directory on the client.

Example:

Client 1

Local Directory:

/mnt/shared

Remote File System:

Server:/shared

After mounting:

/mnt/shared → Server:/shared

The client can now access remote files through the mounted location.

8.6 Mount Table

Each client maintains a mount table.

Example:

Client	Local Mount Point	Remote Location	Status
Client 1	/mnt/shared	Server:/shared	Mounted
Client 2	/mnt/data	Server:/data	Mounted
Client 3	/mnt/project	Server:/project	Unmounted

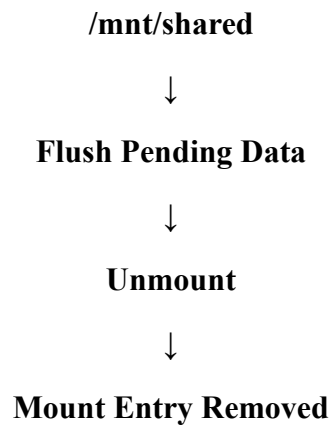
8.7 Unmount Operation

Unmounting disconnects the remote file system from the client.

Before unmounting, the system should verify:

1. The mount point exists.
2. The file system is currently mounted.
3. No active operation is using the mount.
4. Pending writes are completed.

Example:



8.8 Invalid Mount/Unmount Operations

The system should detect invalid operations.

Invalid Mount

Attempting to mount an already-mounted file system:

Error: File system already mounted.

Invalid Unmount

Attempting to unmount a location that is not mounted:

Error: Mount point not found.

Active File Operation

Attempting to unmount while files are actively being written:

Error: Mount point is busy.

8.9 File Operations

The simulator supports the following remote operations:

Create

Client → Create Request → Server → File System → Disk

Open

The server verifies that the file exists and returns an appropriate file handle or identifier.

Read

Client → Read Request → Server → Disk/Cache → Client

Write

Client → Write Request → Server → File System → Disk

Delete

Client → Delete Request → Server → File System → Release Storage

8.10 Simultaneous File Access

Multiple clients may attempt to access the same file.

Example:

Client 1 → Read report.txt

Client 2 → Read report.txt

Client 3 → Write report.txt

Multiple reads can normally occur simultaneously, but concurrent writes require additional control.

8.11 Concurrent Write Management

The advanced challenge is to prevent conflicting writes.

The proposed solution is a **file-locking mechanism**.

Two basic lock types can be used:

Shared Lock

Used for reading.

Multiple clients can hold shared locks simultaneously.

Client 1 → READ

Client 2 → READ

Client 3 → READ

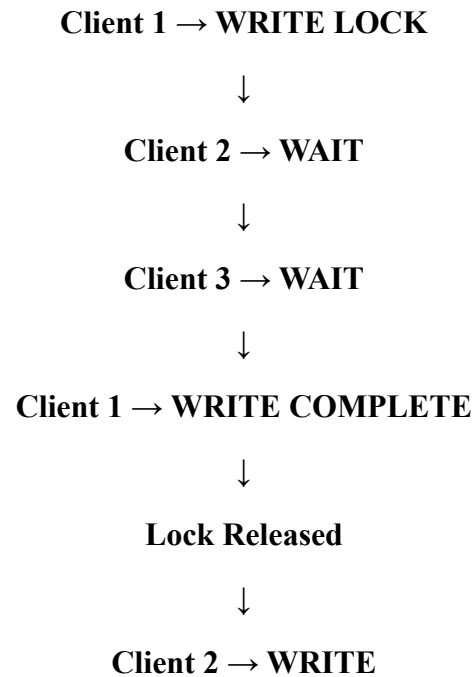
All can access the file concurrently.

Exclusive Lock

Used for writing.

Only one client can hold an exclusive write lock at a time.

Example:



8.12 Conflict Prevention

Suppose two clients attempt to modify the same file.

Without Locking

Client 1 reads version 5

Client 2 reads version 5

Client 1 writes version 6

Client 2 writes version 6

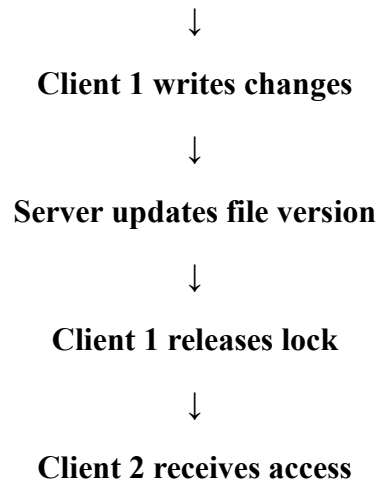
Client 2's update may overwrite Client 1's changes.

With Locking

Client 1 requests write lock



Client 1 modifies file



This prevents simultaneous conflicting writes.

8.13 File Versioning

The system can maintain a version number for each file.

Example:

report.txt – Version 5

After Client 1 writes:

report.txt – Version 6

If Client 2 attempts to update an outdated version, the server can detect the conflict.

Example:

Client Version: 5

Server Version: 6

Result: Write Conflict Detected

The client must refresh the file before performing another write.

8.14 Server Failure Simulation

The simulator should support two server states:

ONLINE

OFFLINE

When the server is online:

Client → Network → Server → File System → Disk

When the server fails:

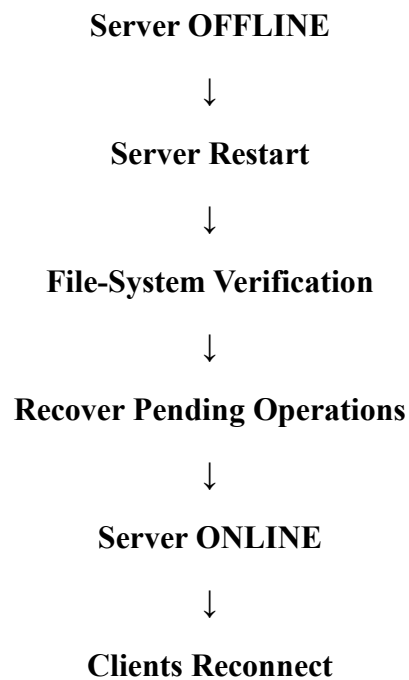
Client → Network → Server UNAVAILABLE

The system should display:

ERROR: Remote file server unavailable.

8.15 Server Recovery

After recovery:

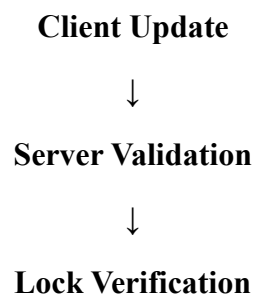


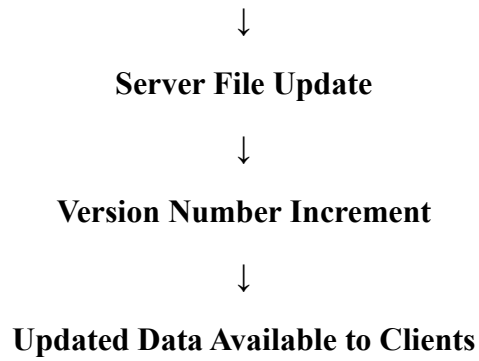
The system can also maintain logs of operations that occurred before the failure.

8.16 Consistency Management

To maintain consistency between clients and the server, the server should act as the authoritative source of file data.

Whenever a client performs a write:

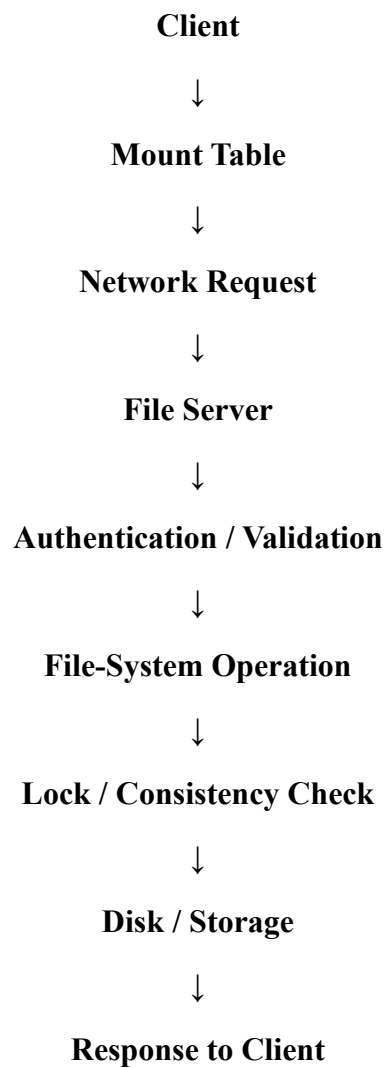




This prevents different clients from permanently maintaining conflicting versions of the same file.

8.17 Complete System Flow

The complete architecture can be represented as:



8.18 Example Scenario

Suppose three clients are connected to the same server.

Step 1 – Mount

Client 1 mounts:

Server:/shared → /mnt/shared

Client 2 mounts:

Server:/shared → /mnt/shared

Step 2 – Read

Both clients read:

report.txt

Since both are reading, shared access is permitted.

Step 3 – Write

Client 1 requests an exclusive write lock.

Client 1 → WRITE LOCK GRANTED

Client 2's write request is temporarily blocked.

Step 4 – Update

Client 1 modifies:

report.txt

The server updates the file and increments its version.

Step 5 – Release

Client 1 releases the lock.

Client 2 can now obtain the write lock and perform its operation.

8.19 Expected Dashboard

The system can display:

SERVER STATUS

Server: ONLINE

Connected Clients: 3

Mounted File Systems: 2

Active Operations: 4

Locked Files: 1

CLIENT STATUS

Client	Connection	Mount Status	Active Operation
Client 1	Connected	Mounted	Writing
Client 2	Connected	Mounted	Reading
Client 3	Connected	Mounted	Idle

FILE STATUS

File	Version	Lock Status	Access
report.txt	6	Exclusive	Client 1
data.csv	3	Shared	Client 2
project.doc	2	Free	None

8.20 Performance Metrics

The simulator can measure:

- Number of client requests.
- Successful operations.
- Failed operations.
- Network requests.
- Average response time.
- Cache hits.
- Concurrent access count.
- Lock wait time.
- Server downtime.
- Recovery time.
- File conflicts prevented.

8.21 Expected Outcome

The final simulator should demonstrate the complete distributed file-system architecture:

Client → Network → File Server → File System → Disk

It should allow users to:

- Create files.
- Open files.
- Read files.
- Write files.
- Delete files.
- Mount remote file systems.
- Unmount remote file systems.
- Manage multiple clients.
- Handle simultaneous access.
- Prevent conflicting writes.
- Simulate server failures.
- Recover from server failures.
- Monitor file consistency.

8.22 Conclusion

The **Distributed Network File System with Mount/Unmount Management** demonstrates how multiple clients can securely and consistently access centralized remote storage.

The simulator combines important operating-system concepts such as **file operations, mounting, unmounting, client-server communication, concurrency, locking, failure handling, recovery, and consistency management.**

The use of shared and exclusive locks prevents conflicting writes, while file versioning provides an additional mechanism for detecting stale updates.

Final System Flow:

Client → Network → File Server → File System → Disk

9. INTELLIGENT FILE-SYSTEM RECOVERY AND CRASH CONSISTENCY SYSTEM

Industry Context

Database Servers / Banking / Mission-Critical Systems

9.1 Problem Statement

File systems used by banking systems, database servers, and other mission-critical applications must remain consistent even when unexpected system failures occur.

A crash can happen while a file is being created, deleted, allocated, or modified. If some operations are completed while others are only partially completed, the file system may enter an inconsistent state.

The proposed **Intelligent File-System Recovery and Crash Consistency System** is a simulator that intentionally generates crashes during file-system operations, identifies resulting inconsistencies, and applies recovery techniques to restore the system to a valid state.

9.2 Objectives

The main objectives of the system are:

1. To simulate common file-system operations.
2. To maintain file metadata using inodes.
3. To maintain free-block information.
4. To simulate unexpected system crashes.
5. To detect file-system inconsistencies.
6. To identify lost blocks.
7. To detect duplicate block allocation.
8. To identify invalid inode references.
9. To detect orphan files.
10. To detect corrupted free-space information.
11. To implement recovery mechanisms.
12. To restore the file system to a consistent state.
13. To compare different recovery techniques.
14. To measure data loss and recovery time.
15. To calculate the consistency level before and after recovery.

9.3 Basic System Architecture

The simulator can be organized into the following components:

File Operation Manager

↓

Metadata Manager

↓

Block Allocation Manager

↓

Crash Generator

↓

Consistency Checker

↓

Recovery Engine

↓

File-System Verification

9.4 File-System Representation

The simulated file system maintains three major structures:

A. Inode Table

The inode table stores file metadata.

Example:

Inode	File Name	Size	Blocks	Status
1	report.txt	3 KB	10, 11, 12	Valid
2	data.csv	4 KB	20, 21, 22, 23	Valid
3	log.txt	2 KB	30, 31	Valid

B. Free-Block Map

The free-block map records whether each disk block is available.

Example:

Block	Status
0	Free
1	Used
2	Used
3	Free
4	Used

A simple representation can be:

0 = Free

1 = Allocated

C. File Directory

The directory maintains the relationship between file names and inode numbers.

Example:

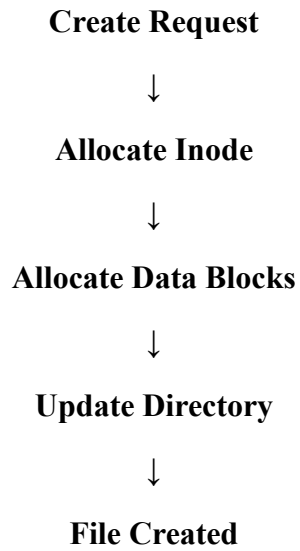
File Name	Inode
report.txt	1
data.csv	2
log.txt	3

9.5 Simulated File Operations

The simulator supports four major operations.

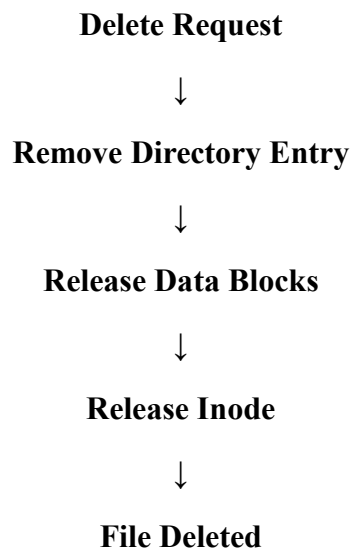
A. File Creation

Process:



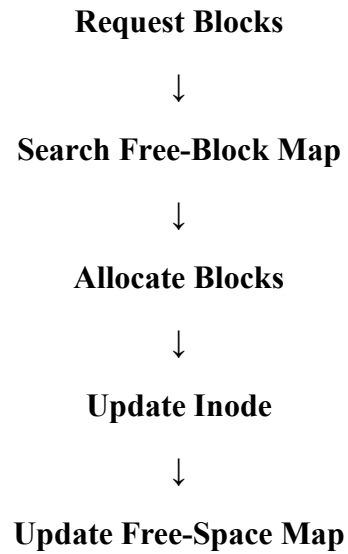
B. File Deletion

Process:



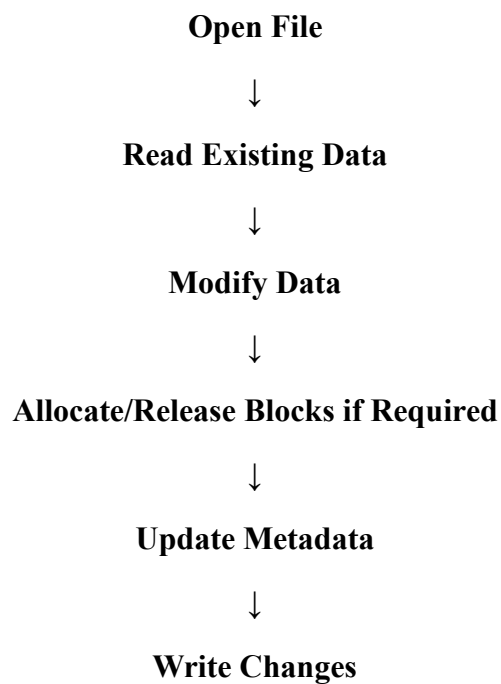
C. Block Allocation

When a file requires additional storage:



D. File Modification

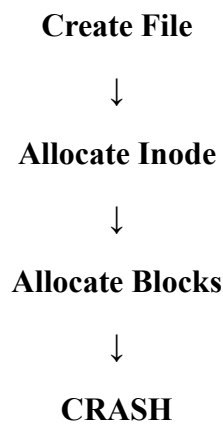
When a file is modified:



9.6 Crash Simulation

The system randomly generates a crash during an operation.

For example:



The operation stops before all metadata is updated.

This may leave the file system in an inconsistent state.

9.7 Crash Scenarios

The simulator can generate different crash scenarios.

Scenario 1 – Crash During File Creation

The inode may be allocated, but the directory entry may not be created.

Result:

Allocated Inode → No Directory Entry

This may create an orphan inode or orphan file.

Scenario 2 – Crash During Block Allocation

A block may be marked as allocated, but the inode may not contain a reference to it.

Result:

Allocated Block → No File Reference

This creates a lost block.

Scenario 3 – Crash During File Deletion

The directory entry may be removed while some data blocks are still marked as allocated.

Result:

File Entry Removed → Blocks Still Allocated

This can result in lost storage space.

Scenario 4 – Crash During Modification

A file may be partially updated before the crash.

Result:

Old Metadata + New Data

or

New Metadata + Incomplete Data

This can lead to file corruption.

9.8 Consistency Problems

The simulator detects the following problems.

A. Lost Blocks

A lost block is allocated according to the free-space map but is not referenced by any file.

Example:

Block 50 = Allocated

but

No inode references Block 50

Therefore:

Block 50 → LOST BLOCK

B. Duplicate Block Allocation

A single physical block is referenced by multiple files.

Example:

File A → Block 75

File B → Block 75

The same block is therefore allocated to two different files.

Result:

DUPLICATE BLOCK ALLOCATION

This can cause data corruption because one file may overwrite another file's data.

C. Invalid Inode References

An inode may contain a reference to:

- A non-existent block.
- A block outside the disk range.
- A block marked as free.
- An invalid inode number.

Example:

Inode 5 → Block 9999

when the disk contains only blocks 0–999.

Result:

INVALID BLOCK REFERENCE

D. Orphan Files

An orphan file or inode exists without a valid directory reference.

Example:

Inode 8 → report.tmp

but:

Directory → No reference to Inode 8

Result:

ORPHAN INODE DETECTED

E. Corrupted Free-Space Information

The free-space map may disagree with inode information.

Example:

Inode → Block 120

but:

Free-Space Map → Block 120 = Free

This creates an inconsistency.

9.9 Consistency Checker

The consistency checker performs several validation steps.

Check 1 – Inode Validation

Verify that every allocated inode contains valid metadata.

Check 2 – Block Validation

Verify that all referenced blocks exist.

Check 3 – Duplicate Detection

Maintain a list of referenced blocks and check whether any block appears more than once.

Check 4 – Free-Space Validation

Compare inode references with the free-block map.

Check 5 – Directory Validation

Verify that directory entries point to valid inodes.

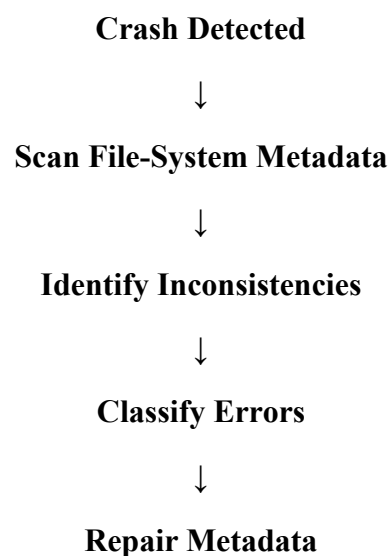
Check 6 – Orphan Detection

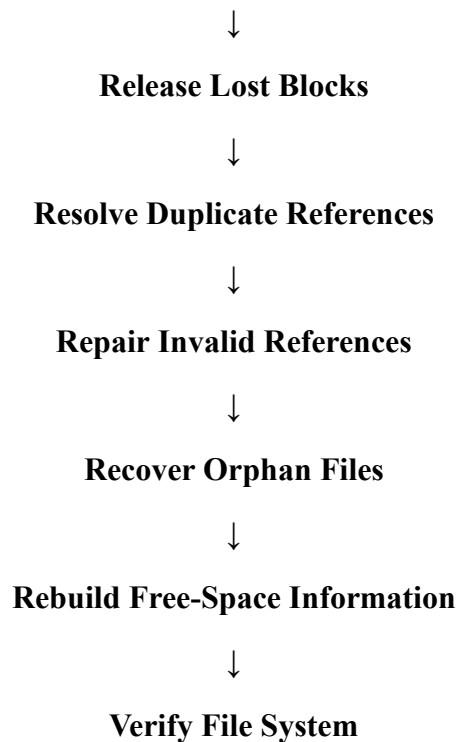
Identify allocated inodes that have no directory reference.

9.10 Recovery Mechanism

Once corruption is detected, the recovery engine attempts to restore consistency.

The recovery process is:





9.11 Recovery Techniques

The simulator can implement multiple recovery approaches.

Approach 1 – Consistency Checking and Repair

The system scans all metadata and repairs inconsistencies.

Advantages:

- Simple to implement.
- Can recover from several types of corruption.
- Suitable for a simulator.

Disadvantage:

- Can require a complete file-system scan.

Approach 2 – Journaling

Before modifying critical metadata, the system records the intended operation in a journal.

Example:

BEGIN TRANSACTION

Create report.txt

Allocate Blocks 10, 11, 12

Update Inode

Update Directory

COMMIT

If a crash occurs before the commit, the incomplete transaction can be rolled back.

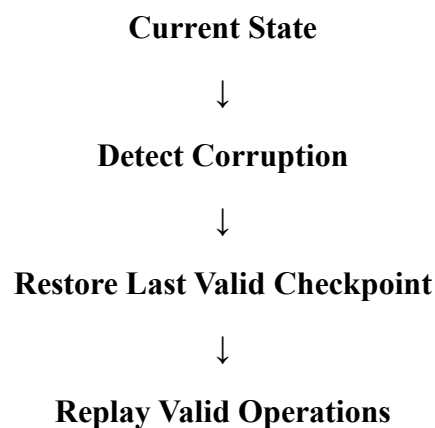
Advantages

- Faster recovery.
 - Reduces inconsistent metadata.
 - Suitable for critical systems.
-

Approach 3 – Checkpoint and Rollback

The system periodically creates a consistent checkpoint.

After a crash:



This reduces recovery time compared with reconstructing the entire file system.

9.12 Recovery Comparison

Recovery Method	Recovery Speed	Data Protection	Implementation Complexity
Consistency Checking	Moderate	Moderate	Low
Journaling	High	High	Medium
Checkpoint & Rollback	High	High	Medium

For mission-critical systems, **journaling or checkpoint-based recovery** can provide better protection than simple consistency checking.

9.13 Data Loss Calculation

The simulator can calculate data loss using:

$$\text{Data Loss} = \text{Number of Unrecoverable Blocks} \times \text{Block Size}$$

Example:

Unrecoverable blocks = 5

Block size = 4 KB

Therefore:

$$\text{Data Loss} = 5 \times 4 \text{ KB} = 20 \text{ KB}$$

9.14 Recovery Time

Recovery time can be measured as:

$$\text{Recovery Time} = \text{Recovery Completion Time} - \text{Crash Detection Time}$$

The simulator can compare recovery times for different techniques.

Example:

Method	Recovery Time
Consistency Check	1.8 seconds
Journaling	0.6 seconds
Checkpoint	0.8 seconds

9.15 Consistency Level

A consistency score can be calculated as:

$$\text{Consistency Level} = \text{Valid Structures} / \text{Total Structures} \times 100$$

For example:

Valid structures = 95

Total structures = 100

$$\text{Consistency Level} = 95\%$$

After successful recovery:

Consistency Level = 100%

9.16 Expected Dashboard

The final system can display:

FILE-SYSTEM HEALTH

Total Files: 250

Allocated Blocks: 7,500

Free Blocks: 2,500

File-System Status: INCONSISTENT

Detected Errors

Lost Blocks: 12

Duplicate Blocks: 2

Invalid References: 3

Orphan Files: 4

Free-Space Errors: 5

RECOVERY STATUS

Recovery Method: Journaling

Data Loss: 8 KB

Recovery Time: 0.72 seconds

Consistency Before Recovery: 91%

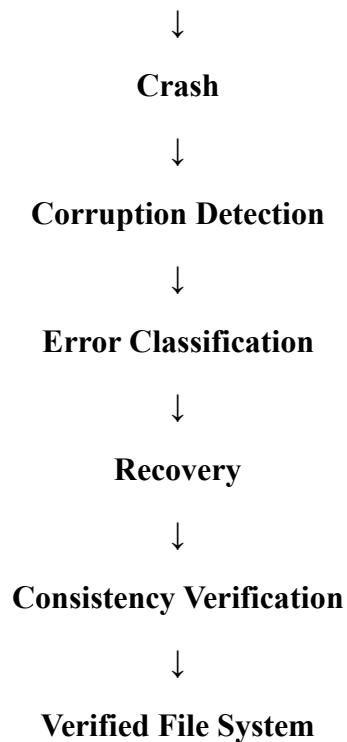
Consistency After Recovery: 100%

File-System Status: HEALTHY

9.17 Expected Outcome

The final system should provide a complete recovery workflow:

Normal State



9.18 Conclusion

The **Intelligent File-System Recovery and Crash Consistency System** demonstrates how operating systems can detect and recover from file-system corruption caused by unexpected failures.

By simulating crashes and identifying lost blocks, duplicate allocations, invalid inode references, orphan files, and free-space inconsistencies, the system provides a practical understanding of file-system reliability.

The comparison of consistency checking, journaling, and checkpoint-based recovery also demonstrates how different recovery techniques affect **data loss, recovery time, and consistency**.

Final System Flow:

Normal State → Crash → Corruption Detection → Recovery → Verified File System

10. HIGH-PERFORMANCE I/O AND FILE-SYSTEM OPTIMIZATION SIMULATOR

Industry Context

Cloud Platforms / Video Streaming / High-Performance Computing

10.1 Problem Statement

Modern cloud platforms, video-streaming services, scientific computing systems, and large enterprise applications generate thousands of file operations.

Poor file allocation, inefficient caching, excessive disk operations, and unsuitable I/O scheduling can significantly reduce system performance.

The proposed **High-Performance I/O and File-System Optimization Simulator** models a large workload and analyzes how different storage and I/O strategies affect performance.

The simulator also includes an adaptive optimization algorithm that dynamically changes system parameters according to workload characteristics.

10.2 Objectives

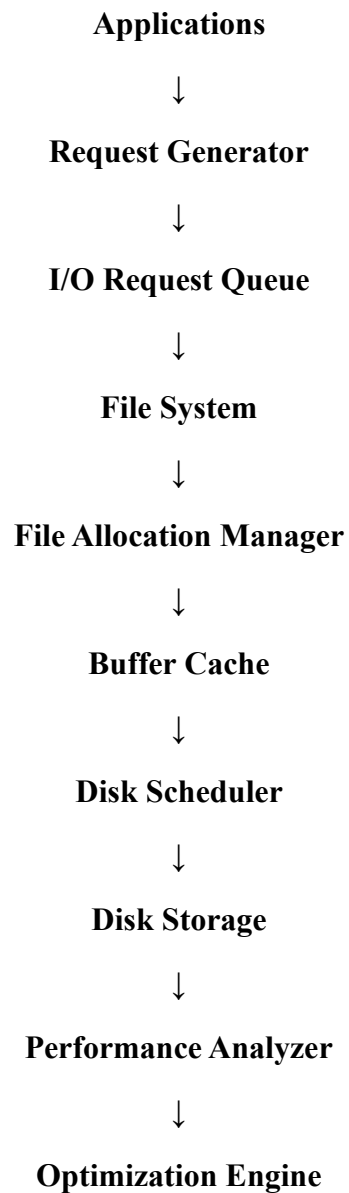
The main objectives are:

1. To generate read/write requests from multiple applications.
2. To simulate different file sizes.
3. To simulate sequential and random access patterns.
4. To implement file allocation.
5. To manage disk blocks.
6. To simulate an I/O subsystem.
7. To maintain request queues.
8. To implement buffer caching.
9. To measure file access time.
10. To measure disk I/O operations.
11. To calculate cache hit ratio.
12. To measure throughput.
13. To calculate storage utilization.
14. To measure fragmentation.
15. To identify system bottlenecks.

16. To implement adaptive workload optimization.

10.3 System Architecture

The proposed simulator can follow this architecture:



10.4 Workload Generator

The workload generator creates file operations from multiple applications.

Each request may contain:

Parameter	Example
Application	Video Server
Operation	READ
File Size	100 MB
Access Pattern	Sequential
Priority	High
Request Time	10 ms

The system can generate thousands of such requests.

10.5 Types of Workloads

A. Video Streaming

Typical characteristics:

- Large files.
- Mostly sequential reads.
- High throughput requirement.
- Repeated access to popular content.

Preferred optimization:

Large Cache + Sequential Access Optimization

B. Database Transactions

Typical characteristics:

- Small and medium-sized operations.
- Frequent random access.
- Read/write mixture.
- Low latency requirement.

Preferred optimization:

High Cache Priority + Efficient Random Access

C. Web-Server Logs

Typical characteristics:

- Continuous writes.
- Sequential file growth.
- Large number of small operations.

Preferred optimization:

Efficient Sequential Allocation + Write Optimization

D. Large Scientific Datasets

Typical characteristics:

- Very large files.
- Sequential and batch access.
- High throughput.
- Large block operations.

Preferred optimization:

Large Blocks + Sequential Scheduling

E. Cloud File Storage

Typical characteristics:

- Mixed workloads.
- Different file sizes.
- Both sequential and random access.
- Highly variable request patterns.

Preferred optimization:

Adaptive Configuration

10.6 File Allocation Module

The simulator can implement multiple allocation methods:

Contiguous Allocation

Linked Allocation

Indexed Allocation

The performance analyzer compares them according to workload.

Example:

Workload	Best Allocation
Sequential Large File	Contiguous
Frequently Growing File	Linked
Random Access Large File	Indexed
Mixed Workload	Adaptive Selection

10.7 Disk-Block Management

The disk is divided into blocks.

Example:

[A][A][A][][B][B][][C][C][][D][D]

The simulator tracks:

- Used blocks.
 - Free blocks.
 - File ownership.
 - Fragmented regions.
 - Allocation history.
-

10.8 I/O Request Queue

Applications generate I/O requests that are placed in a queue.

Example:

Request	Operation	Track
R1	READ	120
R2	WRITE	45

Request	Operation	Track
R3	READ	90
R4	WRITE	150

The I/O subsystem determines the order in which requests are processed.

10.9 Disk Scheduling

The simulator can implement algorithms such as:

- FCFS
- SSTF
- SCAN
- C-SCAN

The performance analyzer compares them using:

- Average seek time.
- Total head movement.
- Request waiting time.
- Throughput.

For example, a heavy sequential workload may benefit from SCAN, while a workload requiring more uniform service time may benefit from C-SCAN.

10.10 Buffer Cache

Frequently accessed blocks are stored in memory.

Example:

Block	Status	Access Count
10	Cached	45
25	Cached	32
40	Cached	8
70	Cached	2

If the requested block exists in the cache:

CACHE HIT

Otherwise:

CACHE MISS → DISK ACCESS

10.11 Cache Hit Ratio

The cache hit ratio is:

$$\text{Cache Hit Ratio} = \text{Cache Hits} / \text{Total Requests} \times 100$$

Example:

Cache hits = 8,500

Total requests = 10,000

$$\text{Cache Hit Ratio} = 85\%$$

A higher cache hit ratio generally reduces physical disk accesses.

10.12 File Access Time

File access time can be estimated using:

$$\text{Access Time} = \text{Seek Time} + \text{Rotational Delay} + \text{Transfer Time}$$

In the simulator, these values can be generated or modeled according to the selected disk configuration.

The system can compare access times across different workloads.

10.13 Throughput

Throughput represents the amount of data processed per unit of time.

Formula:

$$\text{Throughput} = \text{Total Data Processed} / \text{Total Execution Time}$$

Example:

Data processed = 500 MB

Execution time = 5 seconds

$$\text{Throughput} = 100 \text{ MB/s}$$

Higher throughput indicates better system performance for throughput-oriented workloads.

10.14 Storage Utilization

Storage utilization is calculated using:

$$\text{Storage Utilization} = \text{Used Storage} / \text{Total Storage} \times 100$$

Example:

Used storage = 800 GB

Total storage = 1 TB

$$\text{Storage Utilization} = 80\%$$

10.15 Fragmentation Analysis

The simulator measures:

- Internal fragmentation.
- External fragmentation.
- Number of free regions.
- Largest free region.
- Average free-region size.

A high fragmentation level may result in inefficient allocation and increased access time.

10.16 Bottleneck Detection

The performance analyzer identifies the component responsible for reduced performance.

Possible bottlenecks include:

CPU Bottleneck

High request-processing overhead.

Cache Bottleneck

Low cache hit ratio.

Disk Bottleneck

High disk utilization and long access time.

I/O Queue Bottleneck

Large number of waiting requests.

Storage Bottleneck

Insufficient free space or high fragmentation.

10.17 Bottleneck Example

Suppose the simulator produces:

Cache Hit Ratio: 42%

Average Disk Access Time: High

I/O Queue Length: 250

Disk Utilization: 95%

The analyzer may conclude:

Primary Bottleneck → Disk I/O

The system can then recommend:

Increase Cache Size + Optimize Scheduling + Improve Allocation

10.18 Adaptive Optimization Engine

The advanced component dynamically changes system configuration based on workload characteristics.

The optimizer continuously observes:

Workload Pattern

Cache Hit Ratio

Queue Length

Disk Utilization

Fragmentation

Throughput

and then modifies:

- Cache allocation.
- File allocation method.
- I/O scheduling algorithm.
- Block size.

- Request batching.

10.19 Adaptive Decision Rules

Example rules:

Rule 1 – Sequential Workload

If:

Sequential Requests > 70%

then:

Select Sequential-Friendly Allocation + SCAN

Rule 2 – Random Workload

If:

Random Requests > 70%

then:

Prefer Indexed Allocation + Random-Access Optimization

Rule 3 – Low Cache Hit Ratio

If:

Cache Hit Ratio < 50%

then:

Increase Cache Allocation

Rule 4 – High Fragmentation

If:

Fragmentation > 30%

then:

Trigger Storage Reorganization / Compaction Strategy

Rule 5 – Long I/O Queue

If:

Queue Length > Threshold

then:

Change Scheduling Policy or Increase Request Batching

10.20 Adaptive Optimization Algorithm

A simplified algorithm is:

Step 1: Collect workload statistics.

Step 2: Determine whether access is sequential, random, or mixed.

Step 3: Measure cache hit ratio.

Step 4: Measure I/O queue length.

Step 5: Measure fragmentation.

Step 6: Select the appropriate allocation strategy.

Step 7: Select the appropriate scheduling strategy.

Step 8: Adjust cache allocation.

Step 9: Execute the workload.

Step 10: Measure performance.

Step 11: Repeat optimization if performance falls below the target.

10.21 Performance Comparison

The system can generate a comparison table.

Configuration	Access Time	Cache Hit Ratio	Throughput	Fragmentation
---------------	-------------	-----------------	------------	---------------

Configuration A	18 ms	62%	70 MB/s	25%
-----------------	-------	-----	---------	-----

Configuration B	12 ms	78%	95 MB/s	18%
-----------------	-------	-----	---------	-----

Configuration C	9 ms	87%	120 MB/s	12%
-----------------	------	-----	----------	-----

The system can select **Configuration C** as the best-performing configuration for the tested workload.

10.22 Workload-Specific Recommendations

Video Streaming

Workload: Large sequential reads

Recommended Configuration:

Contiguous Allocation + Large Buffer Cache + SCAN

Primary objective:

High Throughput

Database Transactions

Workload: Random reads and writes

Recommended Configuration:

Indexed Allocation + High Cache Priority + Efficient Random-Access Scheduling

Primary objective:

Low Latency

Web-Server Logs

Workload: Continuous sequential writes

Recommended Configuration:

Linked/Sequential-Friendly Allocation + Write Optimization

Primary objective:

Efficient Sequential Writes

Large Scientific Datasets

Workload: Large sequential data processing

Recommended Configuration:

Contiguous Allocation + Large Blocks + SCAN

Primary objective:

Maximum Throughput

Cloud File Storage

Workload: Mixed and unpredictable

Recommended Configuration:

Adaptive Allocation + Adaptive Cache + Adaptive Scheduling

Primary objective:

Balanced Performance

10.23 Expected Dashboard

The final performance analyzer can display:

SYSTEM PERFORMANCE

Total Requests: 50,000

Read Requests: 35,000

Write Requests: 15,000

Average Access Time: 11.5 ms

Cache Hit Ratio: 84%

Disk I/O Operations: 8,200

Throughput: 118 MB/s

Storage Utilization: 76%

Fragmentation: 13%

I/O Queue Length: 24

Detected Bottleneck: None

Recommended Configuration: Adaptive

10.24 Expected Outcome

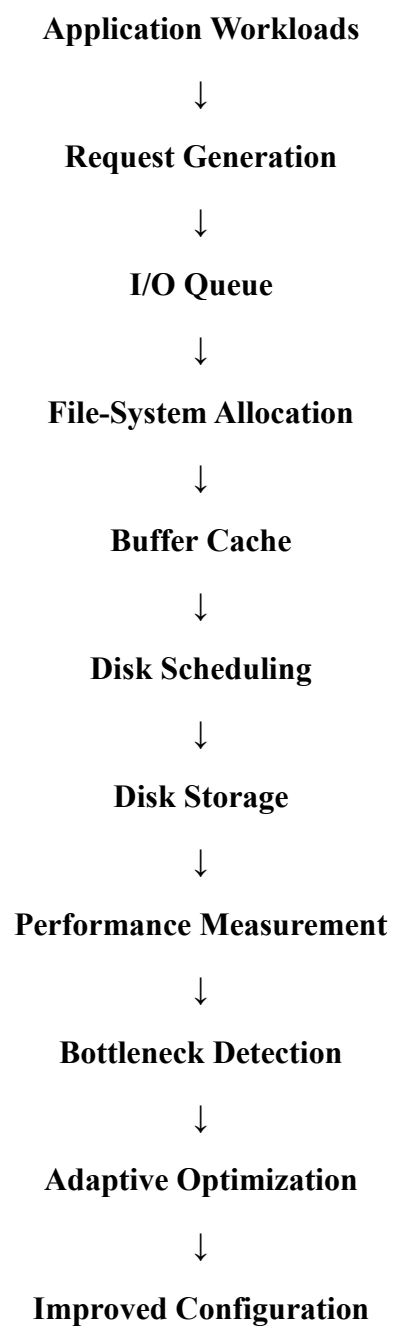
The final system should provide a performance analyzer that:

- Simulates file allocation.
- Simulates disk I/O.
- Manages disk blocks.

- Simulates caching.
 - Measures performance.
 - Detects bottlenecks.
 - Compares configurations.
 - Dynamically optimizes system parameter
-

10.25 Overall System Flow

The complete system can be represented as:



10.26 Conclusion

The **High-Performance I/O and File-System Optimization Simulator** provides a practical environment for studying the relationship between file allocation, caching, disk scheduling, storage utilization, and workload characteristics.

Instead of using one fixed configuration for every workload, the proposed adaptive optimization engine continuously monitors system behavior and selects suitable strategies.

This makes the simulator particularly useful for understanding how different storage environments require different optimization techniques.

Final Optimization Flow:

Workload → Monitor → Analyze → Optimize → Execute → Measure → Adapt
